

API_REFERENCE

Containers API Reference

Package runtime

Import: `digital.vasic.containers/pkg/runtime`

The runtime package provides a unified interface for container operations across Docker, Podman, and Kubernetes. It includes auto-detection logic to select the best available runtime.

Interface ContainerRuntime

```
type ContainerRuntime interface {  
    // Container lifecycle  
    Start(ctx context.Context, containerID string) error  
    Stop(ctx context.Context, containerID string, timeout  
        *time.Duration) error  
    Restart(ctx context.Context, containerID string) error  
    Remove(ctx context.Context, containerID string, force bool)  
        error  
  
    // Container inspection  
    Inspect(ctx context.Context, containerID string)  
        (*ContainerInfo, error)  
    List(ctx context.Context, filters map[string]string)  
        ([]*ContainerInfo, error)  
    Logs(ctx context.Context, containerID string, opts  
        LogOptions) (io.ReadCloser, error)  
  
    // Image operations  
    PullImage(ctx context.Context, image string) error  
    RemoveImage(ctx context.Context, image string, force bool)  
        error  
  
    // Network operations  
    ListNetworks(ctx context.Context) ([]*NetworkInfo, error)  
    CreateNetwork(ctx context.Context, name string, opts  
        NetworkOptions) error  
    RemoveNetwork(ctx context.Context, networkID string) error
```

```

    // Runtime information
    Version(ctx context.Context) (string, error)
    Info(ctx context.Context) (*RuntimeInfo, error)
}

```

Unified interface for container runtime operations.

Struct ContainerInfo

```

type ContainerInfo struct {
    ID          string
    Name        string
    Image       string
    State       ContainerState
    Status      string
    Created     time.Time
    Started     time.Time
    Ports       []PortMapping
    Networks    []string
    Labels      map[string]string
    Environment []string
    Command     []string
}

```

Container information structure.

Fields:

Field	Type	Description
ID	string	Unique container identifier
Name	string	Human-readable container name
Image	string	Container image reference
State	ContainerState	Current state (running, stopped, paused, etc.)
Status	string	Human-readable status message
Created	time.Time	Container creation timestamp
Started	time.Time	Container start timestamp (zero if not started)
Ports	[]PortMapping	Published port mappings
Networks	[]string	Connected network names
Labels	map[string]string	Container labels
Environment	[]string	Environment variables
Command	[]string	Container command

Type ContainerState

```
type ContainerState string

const (
    StateCreated    ContainerState = "created"
    StateRunning    ContainerState = "running"
    StatePaused     ContainerState = "paused"
    StateStopped    ContainerState = "stopped"
    StateRemoving   ContainerState = "removing"
    StateExited     ContainerState = "exited"
    StateDead       ContainerState = "dead"
)
```

Represents container state.

Function AutoDetect

```
func AutoDetect(ctx context.Context) (ContainerRuntime, error)
```

Automatically detects and returns the best available container runtime. Tries Docker first, then Podman, then Kubernetes.

Returns:

- ContainerRuntime -- The detected runtime instance.
- error -- ErrNoRuntimeAvailable if no runtime is found.

Example:

```
runtime, err := runtime.AutoDetect(ctx)
if err != nil {
    log.Fatal("No container runtime available:", err)
}
```

Function NewDockerRuntime

```
func NewDockerRuntime(opts ...DockerOption) (ContainerRuntime,
    error)
```

Creates a new Docker client runtime.

Parameters:

- opts -- Functional options for Docker configuration.

Example:

```
runtime, err := runtime.NewDockerRuntime(
    runtime.WithDockerHost("unix:///var/run/docker.sock"),
```

```
runtime.WithDockerAPIVersion("1.43"),  
)
```

Function NewPodmanRuntime

```
func NewPodmanRuntime(opts ...PodmanOption) (ContainerRuntime,  
error)
```

Creates a new Podman client runtime.

Function NewKubernetesRuntime

```
func NewKubernetesRuntime(opts ...K8sOption) (ContainerRuntime,  
error)
```

Creates a new Kubernetes client runtime.

Package compose

Import: digital.vasic.containers/pkg/compose

The compose package provides Docker Compose orchestration capabilities with profile support and service filtering.

Interface ComposeOrchestrator

```
type ComposeOrchestrator interface {  
    Up(ctx context.Context, opts UpOptions) error  
    Down(ctx context.Context, opts DownOptions) error  
    Restart(ctx context.Context, services []string) error  
    Ps(ctx context.Context) ([]*ServiceStatus, error)  
    Logs(ctx context.Context, service string, opts LogOptions)  
        (io.ReadCloser, error)  
}
```

Interface for Docker Compose operations.

Struct UpOptions

```
type UpOptions struct {  
    ComposeFiles []string  
    Profiles     []string  
    Services     []string  
    Detach       bool
```

```

    Build      bool
    ForceRecreate bool
    Environment map[string]string
}

```

Options for the Up operation.

Fields:

Field	Type	Description
ComposeFiles	[]string	Compose file paths (default: docker-compose.yml)
Profiles	[]string	Profiles to enable
Services	[]string	Specific services to start (empty = all)
Detach	bool	Run in background
Build	bool	Build images before starting
ForceRecreate	bool	Recreate containers even if config unchanged
Environment	map[string]string	Environment variable overrides

Function NewDockerComposeOrchestrator

```

func NewDockerComposeOrchestrator(runtime
    runtime.ContainerRuntime) ComposeOrchestrator

```

Creates a new Docker Compose orchestrator using the provided runtime.

Example:

```

orch := compose.NewDockerComposeOrchestrator(runtime)
err := orch.Up(ctx, compose.UpOptions{
    ComposeFiles: []string{"docker-compose.yml"},
    Profiles:     []string{"production"},
    Detach:       true,
})

```

Package health

Import: digital.vasic.containers/pkg/health

The health package provides multi-strategy health checking with retry policies.

Interface HealthChecker

```
type HealthChecker interface {  
    Check(ctx context.Context, endpoint endpoint.Endpoint) error  
    CheckWithRetry(ctx context.Context, endpoint  
        endpoint.Endpoint, policy RetryPolicy) error  
}
```

Interface for health checking.

Struct RetryPolicy

```
type RetryPolicy struct {  
    MaxAttempts      int  
    InitialInterval  time.Duration  
    MaxInterval      time.Duration  
    Multiplier       float64  
}
```

Configures retry behavior for health checks.

Fields:

Field	Type	De- fault	Description
MaxAttempts	int	3	Maximum retry attempts
InitialInterval	time.Duration	1s	Initial retry interval
MaxInterval	time.Duration	30s	Maximum retry interval
Multiplier	float64	2.0	Exponential backoff multiplier

Function NewDefaultChecker

```
func NewDefaultChecker() HealthChecker
```

Creates a health checker supporting TCP, HTTP, gRPC, and custom checks.

Function TCPCheck

```
func TCPCheck(host string, port int, timeout time.Duration)  
    HealthCheck
```

Creates a TCP health check.

Example:

```
check := health.TCPCheck("localhost", 5432, 5*time.Second)
err := checker.Check(ctx, endpoint.New("postgres", "localhost",
    5432))
```

Function HTTPCheck

```
func HTTPCheck(url string, expectedStatus int, timeout
    time.Duration) HealthCheck
```

Creates an HTTP health check.

Example:

```
check := health.HTTPCheck("http://localhost:8080/health", 200,
    5*time.Second)
```

Function GRPCCheck

```
func GRPCCheck(host string, port int, serviceName string, timeout
    time.Duration) HealthCheck
```

Creates a gRPC health check using the gRPC Health Checking Protocol.

Function CustomCheck

```
func CustomCheck(fn func(context.Context) error) HealthCheck
```

Creates a custom health check from a function.

Example:

```
check := health.CustomCheck(func(ctx context.Context) error {
    // Custom health check logic
    if !isHealthy() {
        return fmt.Errorf("service unhealthy")
    }
    return nil
})
```

Package endpoint

Import: `digital.vasic.containers/pkg/endpoint`

The endpoint package provides service endpoint configuration with builder pattern.

Struct Endpoint

```
type Endpoint struct {  
    Name    string  
    Host    string  
    Port    int  
    Path    string  
    Scheme  string  
}
```

Service endpoint configuration.

Function New

```
func New(name, host string, port int) *Endpoint
```

Creates a new endpoint with default scheme (http) and empty path.

Example:

```
ep := endpoint.New("postgres", "localhost", 5432)
```

Method Endpoint.WithPath

```
func (e *Endpoint) WithPath(path string) *Endpoint
```

Sets the path and returns the endpoint for chaining.

Method Endpoint.WithScheme

```
func (e *Endpoint) WithScheme(scheme string) *Endpoint
```

Sets the scheme (http, https, tcp, etc.) and returns the endpoint for chaining.

Method Endpoint.URL

```
func (e *Endpoint) URL() string
```

Returns the full URL string.

Example:

```
ep := endpoint.New("api", "localhost", 8080).
    WithScheme("https").
    WithPath("/v1/health")
url := ep.URL() // "https://localhost:8080/v1/health"
```

Package lifecycle

Import: digital.vasic.containers/pkg/lifecycle

The lifecycle package provides advanced lifecycle management including lazy boot, idle shutdown, and semaphore control.

Interface LifecycleManager

```
type LifecycleManager interface {
    Start(ctx context.Context, service string) error
    Stop(ctx context.Context, service string, gracePeriod
        time.Duration) error
    Restart(ctx context.Context, service string) error
    GetState(service string) State
    SetLazyBoot(enabled bool)
    SetIdleShutdown(enabled bool, idleTimeout time.Duration)
}
```

Interface for service lifecycle management.

Type State

```
type State string
```

```
const (
    StateUnstarted State = "unstarted"
    StateStarting  State = "starting"
    StateStarted   State = "started"
    StateStopping  State = "stopping"
    StateStopped   State = "stopped"
    StateFailed    State = "failed"
)
```

Lifecycle state enumeration.

Function NewLifecycleManager

```
func NewLifecycleManager(runtime runtime.ContainerRuntime,
    opts ...Option) LifecycleManager
```

Creates a new lifecycle manager.

Options:

- WithLazyBoot(bool) -- Enable lazy boot (on-demand startup)
- WithIdleShutdown(bool, time.Duration) -- Enable idle shutdown
- WithSemaphore(int) -- Set max parallel operations
- WithEventBus(event.EventBus) -- Attach event bus for hooks

Example:

```
lm := lifecycle.NewLifecycleManager(runtime,
    lifecycle.WithLazyBoot(true),
    lifecycle.WithIdleShutdown(true, 30*time.Minute),
    lifecycle.WithSemaphore(5),
)
```

Package monitor

Import: digital.vasic.containers/pkg/monitor

The monitor package provides resource monitoring for system and per-container metrics.

Interface ResourceMonitor

```
type ResourceMonitor interface {
    GetSystemMetrics(ctx context.Context) (*SystemMetrics, error)
    GetContainerMetrics(ctx context.Context, containerID string)
        (*ContainerMetrics, error)
    MonitorThresholds(ctx context.Context, thresholds Thresholds)
        <-chan Alert
}
```

Interface for resource monitoring.

Struct SystemMetrics

```
type SystemMetrics struct {  
    CPUPercent      float64  
    MemoryPercent   float64  
    MemoryUsedGB    float64  
    MemoryTotalGB   float64  
    DiskPercent     float64  
    DiskUsedGB      float64  
    DiskTotalGB     float64  
    Timestamp       time.Time  
}
```

System-wide resource metrics.

Struct ContainerMetrics

```
type ContainerMetrics struct {  
    ContainerID      string  
    CPUPercent       float64  
    MemoryUsedMB     float64  
    MemoryLimitMB    float64  
    MemoryPercent    float64  
    NetworkRxMB      float64  
    NetworkTxMB      float64  
    BlockReadMB      float64  
    BlockWriteMB     float64  
    Timestamp        time.Time  
}
```

Per-container resource metrics.

Function NewResourceMonitor

```
func NewResourceMonitor(runtime runtime.ContainerRuntime)  
    ResourceMonitor
```

Creates a new resource monitor.

Example:

```
monitor := monitor.NewResourceMonitor(runtime)  
metrics, err := monitor.GetContainerMetrics(ctx, "postgres-  
    container")  
if metrics.MemoryPercent > 90.0 {  
    log.Warn("High memory usage:", metrics.MemoryPercent)  
}
```

Package event

Import: digital.vasic.containers/pkg/event

The event package provides a lifecycle event bus for container operations.

Type EventType

type EventType **string**

```
const (  
    EventContainerStarting    EventType = "container.starting"  
    EventContainerStarted    EventType = "container.started"  
    EventContainerStopping    EventType = "container.stopping"  
    EventContainerStopped    EventType = "container.stopped"  
    EventContainerFailed      EventType = "container.failed"  
    EventHealthCheckPassed    EventType = "healthcheck.passed"  
    EventHealthCheckFailed    EventType = "healthcheck.failed"  
    EventResourceThreshold    EventType = "resource.threshold"  
)
```

Container lifecycle event types.

Interface EventBus

```
type EventBus interface {  
    Publish(event Event)  
    Subscribe(eventType EventType, handler func(Event))  
    Unsubscribe(eventType EventType, handlerID string) error  
}
```

Event bus for lifecycle events.

Struct Event

```
type Event struct {  
    ID          string  
    Type        EventType  
    Timestamp   time.Time  
    Container   string  
    Service     string  
    Data        map[string]interface{}  
}
```

Container lifecycle event.

Package discovery

Import: `digital.vasic.containers/pkg/discovery`

The discovery package provides service discovery using multiple strategies.

Interface Discoverer

```
type Discoverer interface {
    Discover(ctx context.Context, opts DiscoveryOptions)
        ([]ServiceInfo, error)
    DiscoverTCP(ctx context.Context, portRange PortRange)
        ([]ServiceInfo, error)
    DiscoverDNS(ctx context.Context, domain string)
        ([]ServiceInfo, error)
}
```

Interface for service discovery.

Struct ServiceInfo

```
type ServiceInfo struct {
    Name      string
    Host      string
    Port      int
    Protocol  string
    Metadata  map[string]string
}
```

Discovered service information.

Function NewDiscoverer

```
func NewDiscoverer(strategies ...Strategy) Discoverer
```

Creates a new service discoverer with specified strategies.

Strategies:

- StrategyTCP -- TCP port scanning
- StrategyDNS -- DNS-based discovery
- StrategyMDNS -- Multicast DNS
- StrategyConsul -- Consul service catalog

Example:

```
disc := discovery.NewDiscoverer(
    discovery.StrategyTCP,
    discovery.StrategyDNS,
)

services, err := disc.Discover(ctx, discovery.DiscoveryOptions{
    HostPattern: "192.168.1.*",
    PortRange:   discovery.PortRange{Start: 5000, End: 6000},
})
```

Package boot

Import: `digital.vasic.containers/pkg/boot`

The boot package provides the high-level BootManager that orchestrates all container operations.

Struct BootManager

```
type BootManager struct {
    // Unexported fields
}
```

High-level orchestration manager composing all container packages.

Function NewBootManager

```
func NewBootManager(opts ...Option) *BootManager
```

Creates a new boot manager with functional options.

Options:

- `WithRuntime(runtime.ContainerRuntime)` -- Set container runtime
- `WithLogger(logging.Logger)` -- Set logger
- `WithHealthCheckRetries(int)` -- Set health check retry count
- `WithParallelStartup(bool)` -- Enable parallel service startup
- `WithLazyBoot(bool)` -- Enable lazy boot
- `WithEventBus(event.EventBus)` -- Attach event bus
- `WithMetrics(metrics.MetricsCollector)` -- Attach metrics collector

Example:

```
manager := boot.NewBootManager(
    boot.WithRuntime(runtime),
```

```
boot.WithLogger(logger),
boot.WithHealthCheckRetries(3),
boot.WithParallelStartup(true),
)
```

Struct ServiceConfig

```
type ServiceConfig struct {
    Name          string
    ComposeFile   string
    ServiceName   string
    HealthCheck   HealthCheck
    Required      bool
    DependsOn     []string
    Env           map[string]string
    Labels        map[string]string
}
```

Configuration for a managed service.

Method BootManager.AddService

```
func (bm *BootManager) AddService(name string, config
    ServiceConfig) error
```

Adds a service to be managed.

Parameters:

- name -- Unique service identifier
- config -- Service configuration

Example:

```
manager.AddService("postgresql", boot.ServiceConfig{
    ComposeFile: "docker-compose.yml",
    ServiceName: "postgres",
    HealthCheck: health.TCPCheck("localhost", 5432,
        5*time.Second),
    Required:    true,
    DependsOn:   []string{},
})
```

Method BootManager.Start

```
func (bm *BootManager) Start(ctx context.Context) error
```

Starts all managed services in dependency order.

Returns: Error if any required service fails to start.

Method `BootManager.Stop`

```
func (bm *BootManager) Stop(ctx context.Context, gracePeriod
    time.Duration) error
```

Stops all managed services in reverse dependency order.

Method `BootManager.HealthCheck`

```
func (bm *BootManager) HealthCheck(ctx context.Context)
    (map[string]HealthStatus, error)
```

Performs health check on all services.

Returns: Map of service names to health status.

Method `BootManager.GetStatus`

```
func (bm *BootManager) GetStatus() *Status
```

Returns current status of all managed services.

Struct `Status`

```
type Status struct {
    Services map[string]ServiceStatus
    Uptime   time.Duration
    Healthy  int
    Unhealthy int
    Total    int
}
```

Overall boot manager status.

Error Types

Common Errors

```
var (  
    ErrNoRuntimeAvailable    = errors.New("no container runtime  
        available")  
    ErrServiceNotFound       = errors.New("service not found")  
    ErrHealthCheckFailed     = errors.New("health check failed")  
    ErrServiceDependency     = errors.New("service dependency  
        failed")  
    ErrInvalidConfiguration  = errors.New("invalid configuration")  
    ErrTimeout               = errors.New("operation timeout")  
)
```

Complete Usage Example

```
package main  
  
import (  
    "context"  
    "log"  
    "time"  
  
    "digital.vasic.containers/pkg/boot"  
    "digital.vasic.containers/pkg/runtime"  
    "digital.vasic.containers/pkg/health"  
    "digital.vasic.containers/pkg/logging"  
)  
  
func main() {  
    ctx := context.Background()  
  
    // Auto-detect runtime  
    rt, err := runtime.AutoDetect(ctx)  
    if err != nil {  
        log.Fatal("No runtime available:", err)  
    }  
  
    // Create logger  
    logger := logging.NewDefaultLogger()  
  
    // Create boot manager  
    manager := boot.NewBootManager(  
        boot.WithRuntime(rt),  
        boot.WithLogger(logger),  
        boot.WithHealthCheckRetries(3),  
        boot.WithParallelStartup(true),  
        boot.WithLazyBoot(false),  
    )
```

```

)

// Add PostgreSQL
manager.AddService("postgresql", boot.ServiceConfig{
    ComposeFile: "docker-compose.yml",
    ServiceName: "postgres",
    HealthCheck: health.TCPCheck("localhost", 5432,
        5*time.Second),
    Required:    true,
})

// Add Redis
manager.AddService("redis", boot.ServiceConfig{
    ComposeFile: "docker-compose.yml",
    ServiceName: "redis",
    HealthCheck: health.TCPCheck("localhost", 6379,
        5*time.Second),
    Required:    true,
    DependsOn:  []string{"postgresql"},
})

// Add API server
manager.AddService("api", boot.ServiceConfig{
    ComposeFile: "docker-compose.yml",
    ServiceName: "api",
    HealthCheck: health.HTTPCheck("http://localhost:8080/
        health", 200, 5*time.Second),
    Required:    false,
    DependsOn:  []string{"postgresql", "redis"},
})

// Start all services
if err := manager.Start(ctx); err != nil {
    log.Fatal("Failed to start services:", err)
}

logger.Info("All services started successfully")

// Health check
statuses, _ := manager.HealthCheck(ctx)
for name, status := range statuses {
    logger.Infof("%s: %s", name, status)
}

// Wait for interrupt...

// Graceful shutdown
manager.Stop(ctx, 30*time.Second)
}

```

Last Updated: February 10, 2026 **Version:** 1.0.0 **Total API**
Methods: 50+ **Status:**  Complete